OPEN AGENT HACKATHON 2026 · TRACK 2 · THE AGENT THAT CAN EXPLAIN WHY

ExplainWhy

An agent that explains *why* your data sources disagree, and answers only with claims a critic has checked against the query results.

NVIDIA Nemotron | Zetaris data harness | Meterless World Model · H-MEM · Markovian

Team gheware (solo) · Rajesh Gheware

github.com/brainupgrade-in/explainwhy

THE PROBLEM

"Why don't these numbers line up?" costs analysts hours

- A revenue series that just **stops** in 2018.
- A join that silently **drops** one company.
- Two figures for the same thing that **don't match**.

The cause is spread across sources: tagging changes, reference-data gaps, duplicated filings, real business events.

A plain LLM is the wrong tool

It answers fluently, invents numbers, and mixes plausible causes with facts. You cannot tell which sentence came from the data.

ExplainWhy works like a careful analyst: hypotheses → checks → ruled out → only what the data shows, with receipts.

WHAT IT DOES

Hypotheses, SQL evidence, a strict critic, a verified answer

Recall
H-MEM brings back prior verified findings, to re-check before use

→

Plan
3–5 competing hypotheses: data artifacts *and* real-world causes

→

Investigate
one bounded step at a time: read-only SQL in Zetaris, every result recorded as evidence E-xxxxxx

→

Critique
citations exist · numbers appear in cited rows · no vacuous zeros · LLM review

→

Answer
verified claims only; causes the data can't prove are labelled "likely"

↺ Unsupported claims go back to the investigator with the critic's reasons (up to 2 repair rounds). Hard limits: 8-minute run deadline, step budget with forced finish, capped retries, single-SELECT guard.

Every role runs on **NVIDIA Nemotron** (`nemotron-3-super-120b-a12b`). The full trace is the World Model event log: question, hypotheses, each query and its rows, each critic verdict, the answer.

REAL RUN · SAMPLE 2

Why does IBM drop out of the SIC join?

```
$ python -m explainwhy --sample 2
[planner]  5 hypotheses: whitespace · zero-padding · code missing from reference · NULL · duplicate rows
[sql]      E-c38e8f  IBM's sic_code in company_profiles              → 1 row (3570)
[sql]      E-84b0e6  more than one row for IBM?                     → 1 row
[sql]      E-1943cf  sic_code = '3570' in sic_codes_table           → 0 rows
[sql]      E-a9171e  length before / after trim                     → 4 / 4
[sql]      E-be1c50  zero-padded variants ('03570')                 → 0 rows
[sql]      E-1943cf  repeated query refused
[critic]   round 1: "the whole table is zero-padded" PARTIAL → sent back for proof
[critic]   round 2: 5 of 5 claims SUPPORTED · hypotheses: 1 supported, 4 refuted

IBM's EDGAR profile lists sic_code 3570, but the company_dns SIC reference table contains no entry
for 3570 or any zero-padded variant, so the join drops IBM. This likely indicates 3570 is not in
the reference list (possibly obsolete).
  ✓ IBM's sic_code in edgar.company_profiles is 3570.                    [E-c38e8f]
  ✓ A search for '3570' and zero-padded variants returns zero rows.   [E-1943cf, E-be1c50]
  ✓ IBM's sic_code has no leading or trailing whitespace.                [E-a9171e]
```

Data: SEC EDGAR submissions + company_dns SIC list — two independent public sources

TRUST

The critic decides what you get to read

Deterministic checks (no model)

- Every claim cites evidence that was actually recorded.
- Every year and 4+-digit number in a claim appears in the cited rows or SQL.
- A zero result proves nothing if its filter value matches no rows (`entity_name = 'Apple'` when the table says "Apple Inc.").

Then an LLM review

- Do the cited rows, and only those, show the claim? supported / partial / unsupported.
- Rejected claims are sent back with reasons; the investigator gathers more evidence or drops them.
- The final summary is written from verified claims only; a new number in it is rejected.

Caught during today's development runs: invented SIC codes (3599, 3500) not in any row, a "no rows" claim built on a misspelled company name, and partial claims sent back for proof.

SPONSOR TECHNOLOGY

Where each sponsor sits in the build

SPONSOR	HOW IT IS USED
Zetaris	**Verified live** (local Zetaris, Docker). Evidence is Lightning SQL over the REST API against 3 live REST sources: SEC XBRL facts, SEC submissions, company_dns SIC list. Our USL `explainwhy_usl` links them with PK/FK rules; `SHOW DQ` flags exactly IBM / SIC 3570 — the agent's answer, reached independently. ZETARIS.md · zetaris/
Meterless	**World Model**: event-sourced investigation state and trace. **H-MEM**: verified findings with source, provenance, confidence and audit ledger, recalled next time. **Markovian**: bounded steps with the COMPLETED / REMAINING / DECISIONS / CONTEXT carryover. engines/
NVIDIA	Nemotron 3 Super 120B runs every role: planner, investigator, critic, writer. llm.py
Cursor	Not used: HackOS Cursor access was still "TBC" on 27 Oct. Built with Claude Code (allowed by the AI Usage Policy).

RESULTS

Three real anomalies in public data, explained with evidence

QUESTION	VERIFIED ANSWER	RUN
Why does Apple's Revenues history stop at FY2018?	0 rows after FY2018; the ASC 606 concept runs FY2019 on. ASC 606 adoption given as the *likely* cause.	60 s · 3 steps
Why does IBM drop out of the SIC join?	IBM's SIC 3570 is absent from the reference list; whitespace, NULLs, zero-padding and duplicate rows ruled out.	71 s · 9 steps · 2 critic rounds
Why does Tesla use two revenue concepts?	Only Revenues in FY2011–2017; both from FY2018, in the same filings, sometimes identical (duplicate tagging).	47 s · 5 steps

Eval 5/6. Live on Zetaris: the IBM question in 85 s, 4 verified claims. Baseline audit: Nemotron alone also scores 5/6 on keywords, but states IBM's SIC code is "NULL" and invents amended filings; every ExplainWhy claim is backed by rows it queried. Markovian context: 42–73 % fewer prompt tokens.

What is not done yet

- Recorded samples and the eval use an offline mirror of the Zetaris views so they replay without keys; one run is live.
- Asked which rows break the USL's DQ rules, the agent invents its own rules instead of reading `SHOW DQ`.
- The critic is strict on numbers and citations, but can still accept a weak interpretation worded as a fact.
- Small, bounded world: 3 companies, 2 revenue concepts.

Next

- An agent-readable USL catalog + DQ endpoint (our Zetaris feature request), so the critic cites DQ results directly.
- H-MEM "dreaming": propose general rules from many verified findings, with human approval.
- A small web UI that shows each claim beside its rows.

TRY IT

Runs in minutes, no GPU, standard-library Python

```
# offline replay of the recorded runs, no keys needed
SAMPLE_MODE=true python -m explainwhy --sample 1

# live NVIDIA Nemotron on the local data mirror
python -m explainwhy "Why does Tesla report revenue under two different XBRL concepts?"

# live Nemotron + live Zetaris (falls back to the mirror, and says so, if unreachable)
python -m explainwhy --zetaris --sample 2

# offline tests with a scripted model
python -m pytest -q
```

Each run writes report.md (answer, verified and dropped claims, hypotheses, every query with its rows), result.json, events.jsonl (the trace) and trace.log. All code was written on 27 Oct 2026; no pre-existing components.